

引用类型

在判断哪些对象需要回收的时候，我们采用了 **引用计数法** 和 **可达性分析**，无论是引用计数法判断对象的引用数量，还是可达性分析中判断对象的引用链，都用到了 **引用** 二字

在 JDK 1.2 中对引用定义过一个狭隘的概念：当一个 Reference 类型的数据中存储的数值是另一块内存的起始地址，我们就称存在引用；但这种定义过于狭隘，它只能描述引用和被引用 2 种状态，而无法描述可有可无的对象，所以在 JDK 1.2 后 Java 对引用类型进行了扩充（4 个引用类型）

- **强引用 Strong Reference**
- **软引用 Soft Reference**：主要为了解决 OOM 异常
- **弱引用 Weak Reference**：主要为了避免内存泄露
- **虚引用 Phantom Reference**：主要为了监听对象的回收

软引用和弱引用的区别

- 如果只想避免 OOM 异常，可以使用软引用
- 如果对应用性能更加在意，想尽快回收一些占用内存比较大的对象，可以使用弱引用（弱引用 **不会影响对象的正常回收**）
- 还可以根据对象是否经常使用来判断：如果频繁使用，尽量使用软引用；如果对象不被使用的可能性更大，可以使用弱引用

内存溢出可以使用 `SoftReference`，内存泄露可以使用 `WeakReference`，GC 监控可以使用 `PhantomReference`

强引用 (Strong Reference)

回收机制：如果一个对象具有强引用，那垃圾回收器绝不会回收它

- 当内存空间不足，Java 虚拟机宁愿抛出 `OutOfMemoryError` 错误，使程序异常终止，也不会靠随意回收具有强引用的对象来解决内存不足的问题
- `object=null`：弱化强引用
- 注意 `数组`：数组是一个特殊的对象

数组：强引用

```
arr[] arrs;  
// 强引用仍然存在  
arrs = null;  
// 强引用没有  
for (int i = 0; i < size; i++)  
    arrs[i] = null;
```

软引用 (Soft Reference)

回收机制：如果一个对象只具有软引用，只要内存空间足够，垃圾回收器就不会回收它；**只有内存空间不足**，才会回收这些对象的内存。

- 一般用于那些，**频繁使用的大内存对象**
- 主要是为了 **解决内存溢出问题**

引用队列：软引用可以和一个引用队列（**ReferenceQueue**）联合使用，如果软引用所引用的对象被垃圾回收器回收，Java 虚拟机就会把这个软引用加入到与之关联的引用队列中

软引用的使用

```
String str=new String("abc");  
// 强引用  
SoftReference<String> softRef=new SoftReference<String>(str);  
    // 软引用  
String str = sfotRef.get();  
// 获取软引用中的对象  
  
ReferenceQueue<Object> referenceQueue = new ReferenceQueue<>  
( );    // 引用队列  
SoftReference softReference = new SoftReference<>(str,  
referenceQueue);  
Reference<?> poll = referenceQueue.poll();
```

弱引用 (Weak Reference)

回收机制：在垃圾回收期扫描到的时候，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存（扫描到即回收）

- 主要是为了 解决内存泄漏问题：如果这个对象是偶尔的使用，并且希望在使用时随时就能获取到，但又 不想影响正常对象的回收，那么你应该用 Weak Reference 来记住此对象
- 如果一个对象只有一条引用链，并且该引用链的末端是一个弱引用，那么我们称之为 弱可达

引用队列：弱引用可以和一个引用队列（ReferenceQueue）联合使用，如果弱引用所引用的对象被垃圾回收，Java 虚拟机就会把这个弱引用加入到与之关联的引用队列中

弱引用的使用

```
String str = new String("abc");  
// 强引用  
WeakReference<String> weakRef = new WeakReference<String>  
(str);           // 弱引用  
WeakReference<String> weakRef = new WeakReference<String>  
(str, queue);    // 弱引用
```

虚引用（Phantom Reference）

回收机制：虚引用也称为 幽灵引用/幻影引用，任何时刻都有可能被垃圾回收器回收

- 我们无法通过虚引用来获取一个对象：

`phantomReference.get()` 方法返回的始终是 `null`

- 一个对象的虚引用，不会对其生命周期造成任何影响

虚引用与软引用/弱引用的一个区别在于：虚引用必须和 引用队列（ReferenceQueue）联合使用，软引用和弱引用时非必需的

虚引用的唯一目的，是在系统回收该对象的时候，可以获得一条通知

- 虚引用有一个很重要的用途就是用来做 堆外内存的释放，`DirectByteBuffer` 就是通过虚引用来实现堆外内存的释放的
- 软引用和弱引用也可以用来监听对象的回收，但是它们可以通过 `get` 方法获取，继而转化为一个强引用，而虚引用就没有这些问题

虚引用的使用

```
String res = "虚引用";
ReferenceQueue<String> queue = new ReferenceQueue<>();
PhantomReference<String> phantomReference = new
PhantomReference<>(res, queue);    // 必须和一个引用队列相关联
```

```
// 通过虚引用获取对象回收时的回调
new Thread(new Runnable() {
    @Override
    public void run() {
        while (true) {
            if (queue.poll() != null) {
                Log.d("Tag", "回收了" + referenceQueue.poll().get());
            }
            // get返回的是null
        }
    }
}).start();
```

ReferenceQueue

ReferenceQueue 引用队列，当对象被回收时会加入到引用队列中，我们可以用它监听对象的回收

- **poll()**、**remove()**：移除并返回队列头部元素（Reference 对象）
 - 在队列头部元素是 null 的情况下，poll() 返回 null，remove() 抛出错误
- **peek()**、**element()**：返回队列头部元素
 - 在队列头部元素是 null 的情况下，peek() 返回 null，element() 抛出错误
- **offer()**、**add()**：添加到队列头部

- 在队列满的时候，offer() 返回 false，add() 抛出错误

```
Reference<? extends String> poll = queue.poll();
```